Sebastián León - 202510399

Nicolas Lizarazo Rincon - 202511191

Gramática:

Alfabeto

Σ = {

Letras mayúsculas

"A", "B", "C", "D", "E", "F", "G", "H", "I", "J", "K", "L", "M", "N", "O", "P", "Q", "R", "S", "T", "U", "V", "W", "X", "Y", "Z",

Letras minúsculas

"a", "b", "c", "d", "e", "f", "g", "h", "i", "j", "k", "l", "m", "n", "o", "p", "q", "r", "s", "t", "u", "v", "w", "x", "y", "z",

Dígitos

"0", "1", "2", "3", "4", "5", "6", "7", "8", "9",

Tipos de dato

"Int", "Bool", "String",

Valores booleanos

"True", "False",

Operadores

"+", "-", "*", "/", "**", "%", "<", ">", "<=", ">=", "<>", "=", "and", "or", "neg", "->",

Puntos, paréntesis y símbolos

".", ",", ":", "(", ")", "[", "]", "{", "}", "$", "'", comilla_string,

Espacio en blanco

" ", tabulacion, salto_de_linea,

Palabras reservadas

"object", "extends", "method", "main", "val", "var", "new", "override", "if", "then", "else", "cond", "for", "in", "and", "or", "neg", "True", "False", "Int", "Bool", "String"

}

Tokens

N = {

letra, minuscula, mayuscula, digito, identifier,

int, bool, char_literal, float_literal, literal_string, caracter_string, interpolacion, literal,

modulo, principal,

objetos, objeto, cuerpo_objeto, herencia, crear_o, argumentos,

parametro_constructor, parametros_constructor,

generico, lista_genericos,

a_miembro, a_metodo,

tipo, tipo_b, tipo_o, tipo_g, tipos,

declaracion, declaracion_val, declaracion_var, asignacion,

metodo, parametros, parametro, cuerpo,

expresion, e_or, e_and, e_neg, e_comp, operador_comp, e_aritmetica, termino, potencia, unario, primario,

if_else, cond, caso,

for, iterador, identifiers,

instruccion, instrucciones

}

Inicial

S = {modulo}

Reglas

S -> modulo

Definimos primero:

mayuscula -> ("A"| "B"| "C"| "D"| "E"| "F"| "G"| "H"| "I"| "J"| "K"| "L"| "M"| "N"| "O"| "P"| "Q"| "R"| "S"| "T"| "U"| "V"| "W"| "X"| "Y"| "Z")

minuscula -> ("a"| "b"| "c"| "d"| "e"| "f"| "g"| "h"| "i"| "j"| "k"| "l"| "m"| "n"| "o"| "p"| "q"| "r"| "s"| "t"| "u"| "v"| "w"| "x"| "y"| "z")

letra -> mayuscula | minuscula

digito -> "0"| "1"| "2"| "3"| "4"| "5"| "6"| "7"| "8"| "9"

identifier -> minuscula (letra | digito | "-")*

int -> digito+

bool -> "True" | "False"

float_literal -> digito+ "." digito+

char_literal -> "'" letra "'"

literal_string -> comilla_string (caracter_string | interpolacion)* comilla_string

caracter_string -> cualquier caracter excepto comilla_string y "$"

interpolacion -> "$" "{" expresion "}"

literal -> int | bool | char_literal | float_literal | literal_string

Iniciar programa (módulo):

modulo -> objetos principal

principal -> "method" "main" "{" instrucciones "}"

En cuanto a objetos:

objetos -> objeto*

objeto -> "object" identifier herencia? lista_genericos? ("(" parametros_constructor? ")")? cuerpo_objeto?

parametro_constructor -> ("val" | "var") identifier ":" tipo

parametros_constructor -> parametro_constructor ("," parametro_constructor)*

cuerpo_objeto -> "{" instrucciones "}"

herencia -> "extends" identifier

crear_o -> "new" identifier ("[" tipos "]")? "(" argumentos? ")" cuerpo_objeto?

argumentos -> expresion ("," expresion)*

Acceso a miembro-metodo:

a_miembro -> identifier ("." identifier)*

a_metodo -> a_miembro "(" argumentos? ")"

Ahora con tipos:

tipo -> tipo_b | tipo_o | tipo_g

tipo_b -> "Int" | "Bool" | "String"

tipo_o -> identifier

tipo_g -> identifier "[" tipos "]"

tipos -> tipo ("," tipo)*

Tipo genéricos - estructura de datos como lista:

generico -> ("+" | "-")? identifier

lista_genericos -> "[" generico ("," generico)* "]"

Declaraciones y asignación:

declaracion -> declaracion_val | declaracion_var

declaracion_val -> "val" identifier (":" tipo)? "=" expresion

declaracion_var -> "var" identifier (":" tipo)? "=" expresion

asignacion -> a_miembro "=" expresion

Métodos:

parametro -> identifier ":" tipo

parametros -> parametro ("," parametro)*

cuerpo -> "{" instrucciones "}"

metodo -> "method" identifier "(" parametros? ")" ":" tipo cuerpo | "override" identifier "(" parametros? ")" ":" tipo cuerpo | identifier "." identifier "=" "method" "(" parametros? ")" ":" tipo cuerpo

Expresiones (de menor a mayor precedencia):

expresion -> e_or

e_or -> e_and ("or" e_and)*

e_and -> e_neg ("and" e_neg)*

e_neg -> "neg" e_neg | e_comp

e_comp -> e_aritmetica (operador_comp e_aritmetica)?

operador_comp -> "<" | ">" | "<=" | ">=" | "<>" | "="

e_aritmetica -> termino (("+" | "-") termino)*

termino -> potencia (("*" | "/" | "%") potencia)*

potencia -> unario ("**" potencia)?

unario -> "-" unario | primario

primario -> literal | a_metodo | a_miembro | crear_o | if_else | cond | "(" expresion ")"

Condicional:

if_else -> "if" expresion "then" expresion "else" expresion

cond -> "cond" "{" caso ("," caso)* "}"

caso -> expresion "->" expresion

Ciclo:

for -> "for" "(" iterador "in" expresion ")" cuerpo

iterador -> identifier | "(" identifiers ")"

identifiers -> identifier ("," identifier)*

Instrucciones:

instruccion -> declaracion | asignacion | metodo | a_metodo | if_else | for

instrucciones -> instruccion*

Convención Importante:

Opcional -> ?

Repetición uno o más veces -> +

Repetición cero o más veces -> *

Agrupación -> ()

O -> |

Los elementos terminales deben de ir entre comillas dobles ""

comilla_string es una forma de mencionar las comillas en un string sin tener que ponerlo sobre otras comillas que queda confuso.

Un identifier no puede ser una palabra reservada.